Git & GitHub

© Zippy Nadel 052-7102588

© Zippy Nadel 052-7102588

Git- מערכת ניהול גרסאות

גיט היא מערכת בקרת גרסאות מבוזרת (DVCS – Distributed Version Control System)
המשמשת לניהול ומעקב אחרי שינויים בקוד או במסמכים לאורך זמן.
בגיט, כל גרסה וכל שינוי שנעשו בקוד נשמרים ונרשמים כך שניתן לשחזר או לבדוק
גרסאות קודמות בקלות.
גיט נבנה לעבודה מקומית, כך שגם ללא חיבור לאינטרנט, אפשר להמשיך לפתח את
הקוד בצורה עצמאית ולהשתמש בכלי הניהולי של גיט ללא תלות במערכת חיצונית.

איך גיט עובד?

גיט שומר את הקוד ואת היסטוריית השינויים במאגר לוקלי (repository) כל שינוי נשמר
בתור "commit" כאשר כל commit כולל את ההבדלים שנעשו בקוד.

Git- Introduction

© Zippy Nadel 052-7102588

GitHub - מערכת ענן לניהול פרויקטים

גיטהאב היא פלטפורמה מבוססת ענן המיועדת לניהול ושיתוף פעולה בקוד.

גיטהאב מבוססת על גיט ומספקת יכולות נוספות לניהול פרויקטים באופן צוותי ומרוחק.

בפלטפורמת גיטהאב, כל מאגר לוקלי יכול להיות משותף לכל חברי הצוות, מה שמאפשר לעבוד על אותו קוד גם אם הצוותים עובדים ממקומות שונים בעולם. בעזרת גיטהאב, כל אחד יכול לגשת לגרסאות עדכניות של הקוד, להוסיף שינויים ולהשתמש בכלי ביקורת ובדיקות שהפלטפורמה מציעה.

חשוב להבין שגיטהב היא אחת מתוך מספר פלטפורמות שונות הקיימות בשוק (כגון GitLab, Azure DevOps, Bitbucket ועוד) מטרתן של פלטפורמות אלו היא לספק סביבה שיתופית לעבודה עם גיט, כך שכל חברי הצוות יכולים לעבוד על אותו פרויקט במקביל ולבצע סנכרון בין העותקים המקומיים שלהם לבין המאגר המרכזי בענן.

כמובן שבעבודה על סביבות אחרות יהיו שינויים בממשק המשתמש אך העקרונות נשארים זהים, ולכן חשוב להבין את העקרונות.

GitHub- Introduction

© Zippy Nadel 052-7102588

למה בכלל צריך מערכת בקרת גרסאות?

בקרת גרסאות היא כלי חיוני לכל פרויקט, בעיקר כאשר עובדים בצוות. מערכת בקרת גרסאות, מאפשרת לנהל ולעקוב אחרי כל שינוי בקוד באופן מסודר – בין אם השינוי נעשה ע"י אדם יחיד או על ידי צוות. כאשר משתמשים במערכת בקרת גרסאות, ניתן:

-למנוע טעויות: אם נעשה שינוי שגוי בקוד, קל לחזור לגרסה קודמת.

-לשפר את האיכות: כל שינוי נרשם, כך שאפשר לבצע ביקורת על כל חלק ולהבין את ההשפעה של כל שינוי על הפרויקט כולו.

-להשוות בין גרסאות קודמות: כך נוכל לראות אילו שורות השתנו ולמה.

-לעבוד בצוות: כל חבר צוות יכול לעבוד במקביל על קבצים וענפים שונים, ללא חשש לשיבושים בקוד.

גיט מול מערכות אחרות – מה מיוחד בו?

שלא כמו מערכות בקרת גרסאות מרכזיות, בגיט העבודה היא מבוזרת (לוקלית ומרוחקת), מה שמאפשר גמישות רבה יותר. גיט שומר את כל היסטוריית הקוד באופן מקומי על מחשב המשתמש, כך שתמיד אפשר להמשיך לעבוד ולאחר מכן לסנכרן את השינויים עם מאגר מרוחק כמו גיטהאב.

The Value of Version Control

© Zippy Nadel 052-7102588

גיט- התחלת עבודה והתקנה

הצעד הראשון בשימוש בגיט הוא להוריד ולהתקין את התוכנה: Git Downloads

לאחר ההתקנה, ניתן לוודא שההתקנה בוצעה בהצלחה על ידי הרצת הפקודה (בשורת הפקודה): **git -version**

יש ליצור חשבון חדש (או להכנס לחשבון אם כבר קיים) בגיטהאב.

הגדרת פרופיל משתמש בגיט

חשוב להגדיר את השם והמייל של המשתמש כדי שכל commit יכיל את פרטי המשתמש המבצע. כך ניתן לעקוב אחרי השינויים שנעשו על ידי כל חבר צוות.

על מנת לבדוק את פרטי המשתמש יש להריץ את הפקודה: **git config --list**

אם יש צורך לשנות את פרטי המשתמש- יש להריץ את הפקודות:

git config --global user.name "Your GitHub username"

git config --global user.email "Your GitHub Email"

Git- Getting Started

© Zippy Nadel 052-7102588

תרגיל 1

גיט הגדרת סביבה והתקנה:

הורידי והתקיני את גיט.

פתחי חשבון חדש בגיטהאב (שימי לב בבחירת שם משתמש ואימייל שפרטים אלו ילוו אותך).

קנפגי את פרטי המשתמש שלך בגיט במחשב הלוקלי שלך.

בדקי את הפרטים בעזרת הפקודה: git config --list

Git- Getting Started- Exercise

© Zippy Nadel 052-7102588

יצירת רפוזיטורי חדש

רפוזיטורי (Repository) הוא מאגר שמכיל את הקוד ואת היסטוריית השינויים שלו.

כל פרויקט בגיט מתחיל עם יצירת רפוזיטורי שמאפשר לשמור גרסאות קוד, לבצע שינויים, לנהל אותם ולעבוד עליהם כצוות או באופן אישי.

בגיט, העבודה מתבצעת במקביל בשני סוגי מאגרים:

Local Repository – מאגר מקומי שמאפשר עבודה ישירה על הקוד במחשב האישי.

Remote Repository – מאגר מרוחק שמאוחסן בענן (למשל בגיטהב) ומשמש כמאגר מרכזי לסינכרון קוד בין צוותים או מחשבים שונים.

חשוב לציין שיש להקפיד לסנכרן בין 2 המאגרים על מנת ליצור אחידות ותאימות בין הקוד המקומי והמרוחק.

Creating a New Repository

© Zippy Nadel 052-7102588

רפוזיטורי לוקלי – עבודה עם git init

כדי להתחיל פרויקט בגיט באופן מקומי, יש להשתמש בפקודת git init אשר יוצרת רפוזיטורי לוקלי במחשב האישי.
פקודה זו תיצור תיקיית .git בתיקיית הפרויקט, שבתוכה נשמרים כל המידע והיסטוריית הגרסאות של הפרויקט.

<u>תהליך יצירת הרפוזיטורי:</u>

1. כניסה לספריית הפרויקט: תחילה יש להכנס לתיקייה שבה רוצים לשמור את הפרויקט (לדוגמה בתוך מסמכים/ שולחן העבודה)
2. הפעלת הפקודה git init: פעולה זו תאתחל את גיט ותהפוך את התיקייה לרפוזיטורי מקומי. (יש לכתוב את הפקודה **git init** בשורת הפקודה)

חשוב לוודא שהפקודה נעשית על התיקיה שבה רוצים לשמור את הפרויקט (יש לוודא בשורת הפקודה שאכן נמצאים בתוך אותה תיקיה)

3. בגרסאות ישנות של גיט מומלץ לשנות את השם של ה- Branch הדיפולטי ע"י הפקודה: **git branch -M main**

Creating a Local Repository

© Zippy Nadel 052-7102588

יצירת רפוזיטורי מרוחק בגיטהאב:

1. יש להכנס לחשבון הגיטהאב שלנו ולבחור באפשרות **New Repository.**
2. יש לתת שם לפרויקט, ניתן להוסיף תיאור אם רוצים, וניתן לבחור האם הרפוזיטורי יהיה ציבורי או פרטי. (אם המאגר ציבורי כל אחד יכול לראות את המאגר, אם הוא פרטי יש להוסיף משתמשים על מנת שיוכלו לגשת למאגר)

חיבור בין רפוזיטורי מקומי לגיטהב:

לאחר יצירת רפוזיטורי לוקלי ומרוחק, אפשר לחבר את המאגר המקומי עם המאגר המרוחק, כדי שיהיה ניתן לשתף ולסנכרן את הקוד בענן.

ניתן לעשות זאת בעזרת הפקודה:

git remote add origin https://github.com/<YourUsername>/<YourRemoteRepoName>.git

הפקודה git remote add origin מגדירה את הרפוזיטורי המרוחק כיעד לשליחת הנתונים מהמאגר המקומי.

Creating a Remote Repository on GitHub

© Zippy Nadel 052-7102588

Cloning a Repository

הפקודה git clone משמשת כדי להעתיק רפוזיטורי מרוחק **קיים** (כמו זה שנמצא בגיטהאב) למחשב המקומי.

כאשר עושים Clone למאגר כל הקוד כולל הסטוריית גרסאות והענפים של הרפוזיטורי המרוחק יועתקו למחשב למאגר מקומי, והמאגר מקומי הזה יהיה מסונכרן עם המקור,

בכך גם המאגר המרוחק שממנו נעשה ה- Cloning יהיה היעד לשליחת נתונים של המאגר המקומי.

<u>תהליך Cloning Repository:</u>

1. יש ליצור מאגר מרוחק כמו שנכתב לעיל (בשקופית הקודמת), אין צורך ליצור מאגר מקומי.
2. יש להכנס לתיקייה שבה רוצים לשמור את הפרויקט במחשב האישי ולפתוח שם את שורת הפקודה.
3. הפקודה של Clone: **git clone https://github.com/<YourUsername>/<YourRemoteRepoName>.git**

Cloning a Repository

© Zippy Nadel 052-7102588

Git init

הפקודה **git init** משמשת כדי לאתחל רפוזיטורי חדש במחשב המקומי מאפס. היא יוצרת מאגר מקומי שבו מתחילים לנהל גרסאות, והוא אידיאלי לפרויקטים שמתחילים מהשלב הראשון.

Git clone

הפקודה **git clone** משמשת כדי להעתיק רפוזיטורי מרוחק קיים (כמו זה שנמצא בגיטהאב) למחשב המקומי.

מתי נשתמש בכל אחת מהפקודות?

-יש להשתמש ב- git init כאשר מתחילים פרויקט חדש במחשב מקומי ורוצים להפוך אותו לרפוזיטורי בגיט.

-יש להשתמש ב- git clone כאשר רוצים להוריד עותק של רפוזיטורי קיים (שנמצא בענן) לעבודה מקומית.

Cloning VS Initializing

© Zippy Nadel 052-7102588

בכל מאגר חדש, מומלץ מאוד לכלול שני קבצים בסיסיים:

(במאגר מרוחק בגיטהב, ניתן להוסיף קבצים אלו בצורה אוטומטית בעת יצירת הרפוזיטורי.)

קובץ README

קובץ טקסט המתאר את הפרויקט וכולל מידע בסיסי על מטרתו ואופן השימוש בו.

<u>אופן יצירת הקובץ:</u>

בתיקיה של הרפוזיטורי יש ליצור קובץ שנקרא README.md, ואחרי שנוצר ניתן לערוך אותו בכל עורך טקסט.

קובץ gitignore

קובץ המורה לגיט אילו קבצים יש להתעלם מהם ולא לשמור במאגר, קבצים אלו ישמרו לוקלית אך לא יהיו חלק מניהול הגרסאות ולא יעלו למאגר המרוחק. (כמו קבצים זמניים או קבצי קונפיגורציה ועוד).

<u>אופן יצירת הקובץ:</u>

בתיקיה של הרפזיטורי יש ליצור קובץ הנקרא gitignore. ולהוסיף בכל עורך טקסט את הקבצים שלא רוצים לעקוב אחריהם.

חשוב לציין שביצירת פרויקט ריאקט חדש נוצר אוטומטי קובץ gitignore שמכיל בצורה דיפולטית את כל הקבצים הרלונטיים.

כאשר רוצים להוסיף קובץ gitignore לפרויקט ב- dotnet ניתן לעשות זאת בקלות באמצעות הפקודה:**dotnet new gitignore**

Essential Files for Every Repository

© Zippy Nadel 052-7102588

תרגיל 2

התחלת עבודה עם Repositories:

1. צרי מאגר מקומי ומאגר מרוחק וקשרי בין המאגרים.
2. צרי מאגר מרוחק חדש ועשי לו Clone למחשב לוקלי.

שימי לב, יש להוסיף קבצי GitIgnore & readme אך אין צורך בשלב זה להוסיף אותם לשני המאגרים. (אם יצרת את הקבצים לוקלית, קבצים אלו יעלו למאגר המרוחק כפי שיורחב בהמשך על העלאת קבצים למאגר מרוחק).

התרגול בהמשך יעשה בין השאר על אחד מהמאגרים הנ"ל (לא על שתיהם, עם כל זאת בשביל התרגול יש צורך להתנסות בשני הסוגים), למען הסדר- תני למאגר זה שם demo או שם מייחד אחר...

New Repository -Exercise

© Zippy Nadel 052-7102588

כלים לעבודה עם גיט – שורת הפקודה מול כלים מובנים

את כל הפקודות בגיט ניתן לבצע גם דרך שורת הפקודה וגם באמצעות כלים מובנים שמספקים תוכנות ייעודיות או עורכי קוד רבים לדוגמה ב- Visual Studio Code וב- Visual Studio יש אפשרות להשתמש בלחצני גיט בנוחות ישירות מהממשק, כך שכל פקודה בשורת הפקודה קיימת גם בלחיצה על כפתור.

אין הבדל מהותי בין שימוש בפקודות לשימוש בכפתורים

חשוב להבין שאין באמת הבדל מהותי בין השימוש בלחצני גיט המובנים לבין שימוש ישיר בפקודות בשורת הפקודה – הפעולות הן אותן פעולות, ללא שינוי. לכן, כל אחד יכול לבחור את הכלי שנוח לו ביותר, בהתאם להעדפותיו האישיות ולסביבת העבודה.

המלצה לשימוש בפקודות גיט בשורת הפקודה

למרות שניתן לבצע את כל הפקודות גם באמצעות לחצנים, ההמלצה היא להשתמש ישירות בפקודות בשורת הפקודה או בטרמינל. עבודה דרך שורת הפקודה מעניקה גמישות ושליטה מלאה על כל פעולה ופעולה, וגם מסייעת להכיר את פקודות גיט בצורה ישירה ויסודית יותר. עבור מי ששואף להעמיק ולהתמקצע בתחום, הבנת הפקודות היא יתרון משמעותי.

Git Commands - Command Line vs Built-In Tools

© Zippy Nadel 052-7102588

עבודה ב- Git Bash דרך הטרמינל של VS Code

ההמלצה לדרך עבודה הנוחה ביותר היא לבצע את כל פקודות גיט ישירות מהטרמינל, וליתר נוחות ב- Git Bash המובנה בתוך הטרמינל של עורך הקוד Visual Studio Code.

<u>בצורה זו, ניתן לראות את הקוד והפקודות באותו חלון, מה שמאפשר שליטה מלאה ונוחות מרבית:</u>

- **VS Code**– כעורך הקוד, יש אפשרות לראות את קבצי הקוד וכן הוא מציע תמיכה מובנית בעבודה עם גיט (דרך ה- Source Control.)
- **Git Bash-** מספק סביבה נוחה להקלדת פקודות גיט ישירות בטרמינל.

גם בפרויקטים שמפותחים בסביבות נוספות (למשל, בפרויקטים ב-Net. שמפותחים בד"כ ב- Visual Studio)- ניתן לכתוב את הקוד בסביבה שבה נהוג לפתח את הקוד ורק כאשר רוצים לכתוב פקודות גיט- לפתוח את הפרויקט ב- VS Code ולהשתמש בטרמינל ב- Git Bash.

חשוב להדגיש שכל אחד יכול לבחור את השיטה המתאימה לו – גם עבודה עם כפתורים וממשקי משתמש בנויים, גם שימוש בטרמינל ב- Git Bash וגם כל שילוב אחר הם כולם שיטות תקפות ונכונות.

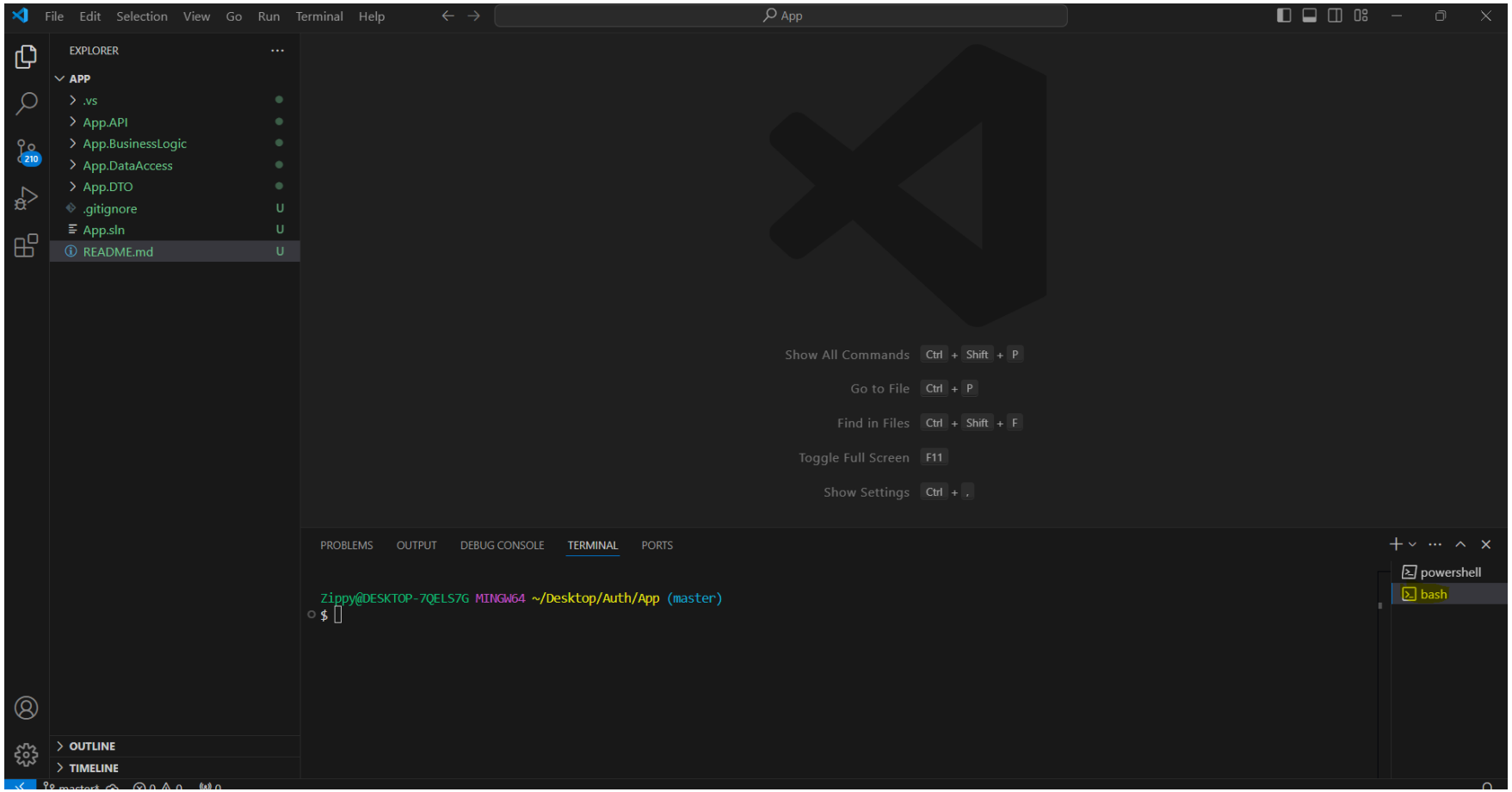
ההמלצה על עבודה ישירה עם הפקודות בטרמינל (Git Bash) היא מתוך הבנה שזה מקדם מקצועיות וגמישות בעבודה.

Working with Git Bash in the Terminal

© Zippy Nadel 052-7102588

דוגמה לפרויקט .Net שהפיתוח ב- Visual Studio

ואילו פקודות גיט יבוצעו דרך ה Git Bash-שמובנה בטרמינל של VS Code



Git Bash in the Terminal

© Zippy Nadel 052-7102588

איך גיט מזהה שינויים בקבצים?

גיט עוקבת אחרי כל הקבצים ברפוזיטורי ומנהלת אותם בשלושה מצבים עיקריים (הרחבה ודוגמאות על כל שלב בהמשך):

1. **Untrack** (לא במעקב): קבצים חדשים שנוספו לתיקייה אך עדיין לא הוספו לגיט. גיט מזהה אותם כקבצים לא מנוהלים.
2. **Modified** (שינוי בקובץ קיים): קבצים שכבר נמצאים במעקב, אך עברו שינוי מאז השמירה האחרונה.
3. **Stage** (מוכן לשמירה): קבצים ששונו ונוספו ל- Staging Area השלב הממתין ל- commit (לשמירה) הבא. בשלב זה הם מוכנים להיכנס להיסטוריית השינויים של הרפוזיטורי.

File Changes and Git’s Tracking System

© Zippy Nadel 052-7102588

שלב 1:

יצירת קובץ חדש - מצב "Untracked"

ברגע שנוצר קובץ חדש בשם Demo File גיט עדיין לא יודע עליו דבר.

מבחינת גיט, זהו קובץ חדש לגמרי, ולכן הוא מוגדר כ-Untracked.

בשלב הזה, גיט רואה שהקובץ קיים בתיקייה, אך אינו עוקב אחר השינויים בו עד שנוסיף אותו למעקב.

- git status בודקת את מצב הרפוזיטורי הנוכחי:

פקודה זו מציגה אילו קבצים השתנו, האם יש קבצים חדשים שנוספו או שינויים שטרם נשמרו.

ניתן לראות את Demo File באדום ברשימת הקבצים בסטטוס של Untracked.

Tracking a File in Git: A Step-by-Step -Untracked File

© Zippy Nadel 052-7102588

דוגמה:

יצירת קובץ חדש בשם Demo file

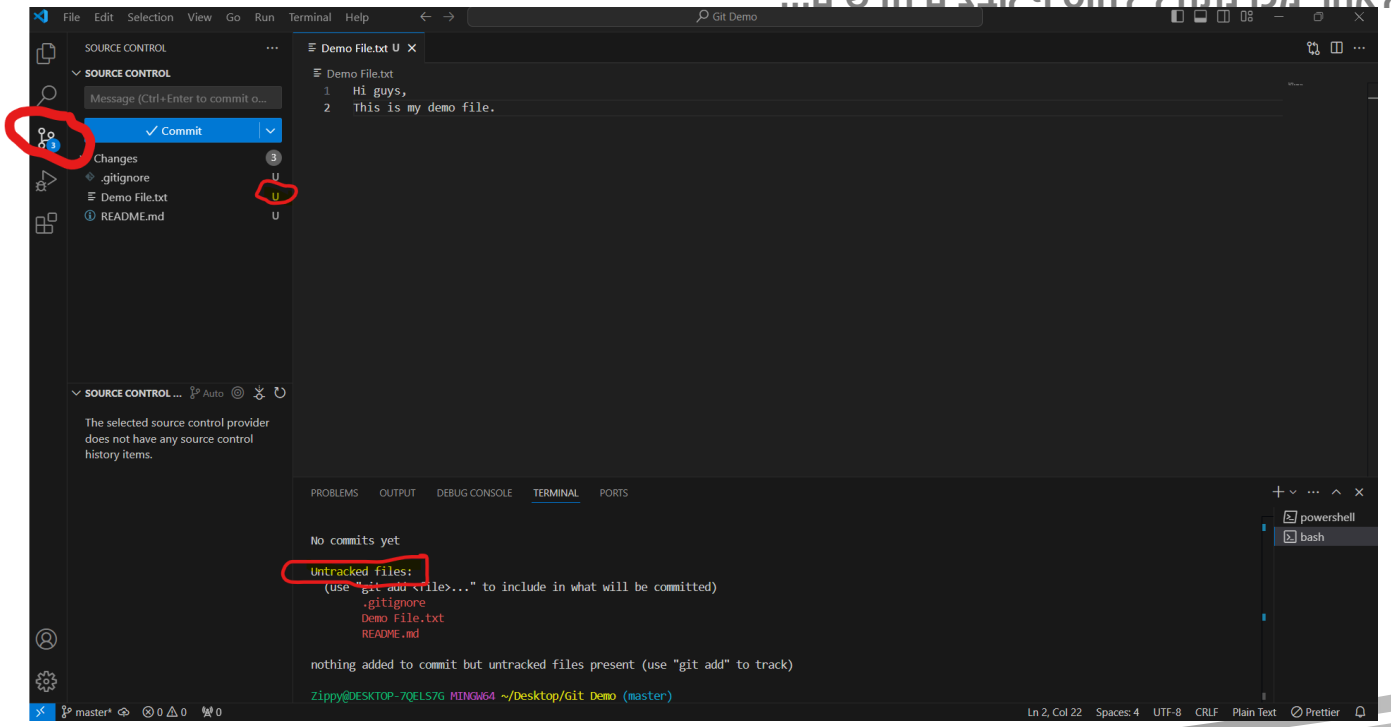
ניתן לראות בטאב של ה- Source Control שהקובץ מסומן ב- U (Untracked)

גם הפקודה Git status מציגה את הקובץ כ- Untracked (באדום).

הערה: מאחר וכרגע מדובר במאגר חדש בכוונה יש שם את הקבצים gitignore & readme

מתוך מטרה להצמד לקבצים שקיימים אצלכן. כמובן שבהנהלות אמיתית נשמור קודם כל את הטמפלייט הבסיסי של הפרויקט כולל

הקבצים הנל ורק לאחר מכן נתחיל להוסיף קבצים חדשים...



Untracked File- Example

© Zippy Nadel 052-7102588

שלב 2:

הוספת הקובץ ל- Staging Area -מצב Staged

כדי להתחיל לעקוב אחרי Demo File ולומר לגיט "אני רוצה שהקובץ הזה יהיה חלק מההיסטוריה של הפרויקט", נשתמש בפקודה git add.

הפקודה הזו מוסיפה את הקובץ ל- Staging Area (שלב ביניים שבו הקובץ ממתין לשמירה)

הפקודה היא: **git add "Your file name"**

לחילופין אם רוצים להוסיף את כל הקבצים שהשתנו ניתן לעשות את הפקודה: **git add .**

בשלב זה הפקודה git status תציג את הקובץ בירוק- Changes to be committed

Tracking a File in Git: A Step-by-Step -Staging File

© Zippy Nadel 052-7102588

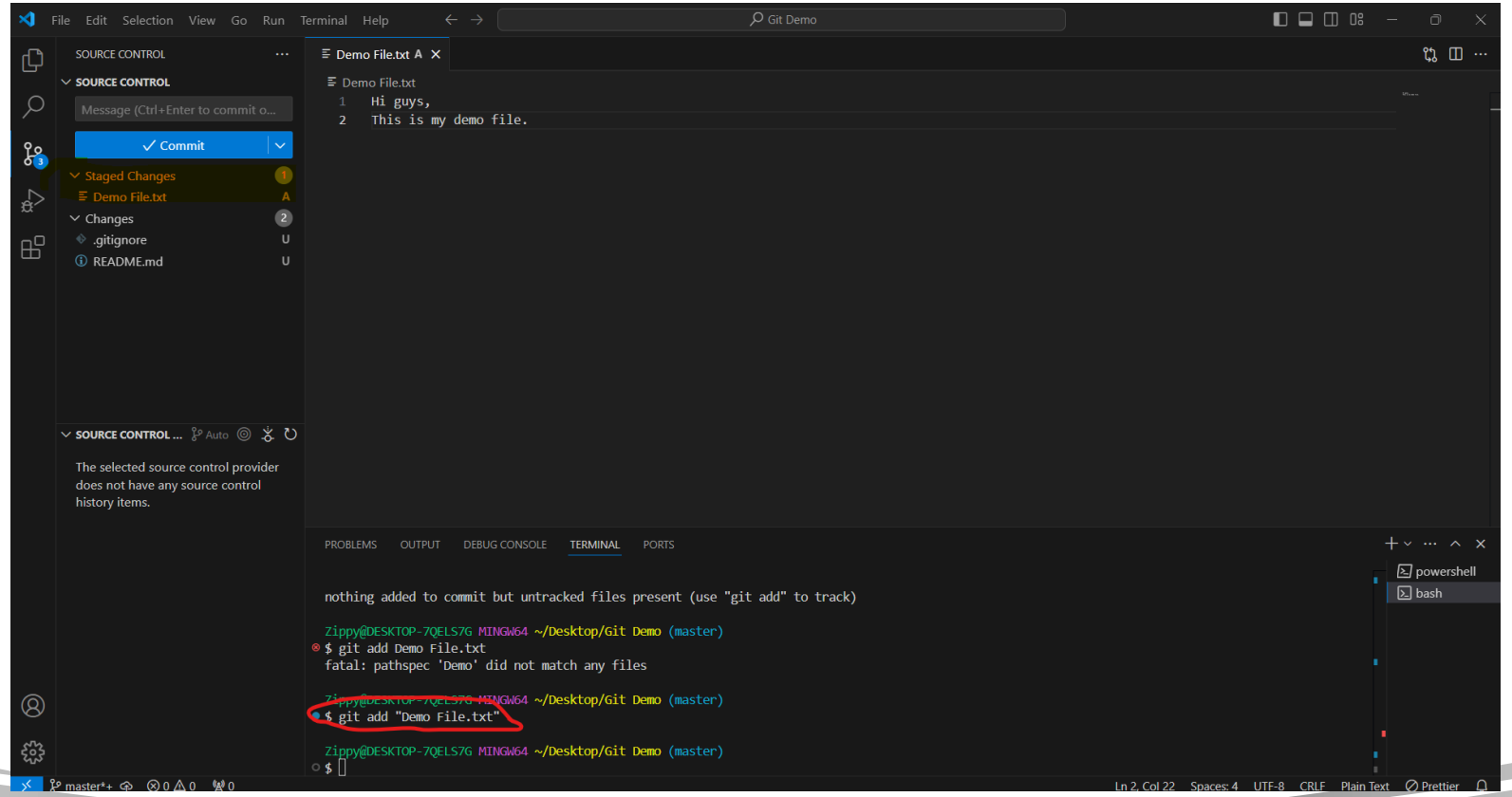
דוגמה:

הוספת הקובץ לשלב ה- Stage באמצעות הפקודה git add

ניתן לראות בטאב של ה- Source Control שהקובץ עבר מה- Changes ל- Staged Changes

גם הפקודה Git status מציגה את הקובץ כ- Staged (בירוק)

הערה:על מנת לפשט תהליכים מאחר וכרגע מדובר במסלולו של הקובץ Demo File כל התהליך נעשה עליו למרות שיש עוד קבצים בפרויקט.



Staged File- Example

© Zippy Nadel 052-7102588

שלב 3:

שמירת הקובץ ב- Commit -מצב Committed

כדי לסיים את התהליך ולשמור את Demo File בהיסטוריה של הרפוזיטורי, נבצע commit.

זהו שלב חשוב שבו גיט שומר את הקובץ כגרסה מסודרת שניתן לחזור אליה בהמשך.

פקודת Commit- **git commit -m "Add Demo File".**

(m- מבטא message, יש לכתוב הודעה ברורה מה מהות ה- Commit)

ברגע שנבצע את הפקודה הזו, Demo File ייכנס להיסטוריית הקוד של הפרויקט, וכל שינוי שנעשה בו ייחשב כגרסה חדשה.

הערה: Commit שומר את כל הקבצים שנמצאים בשלב ה- Stage. קובץ שהשתנה אך לא נוסף ל-Stage- לא יישמר ב-Commit

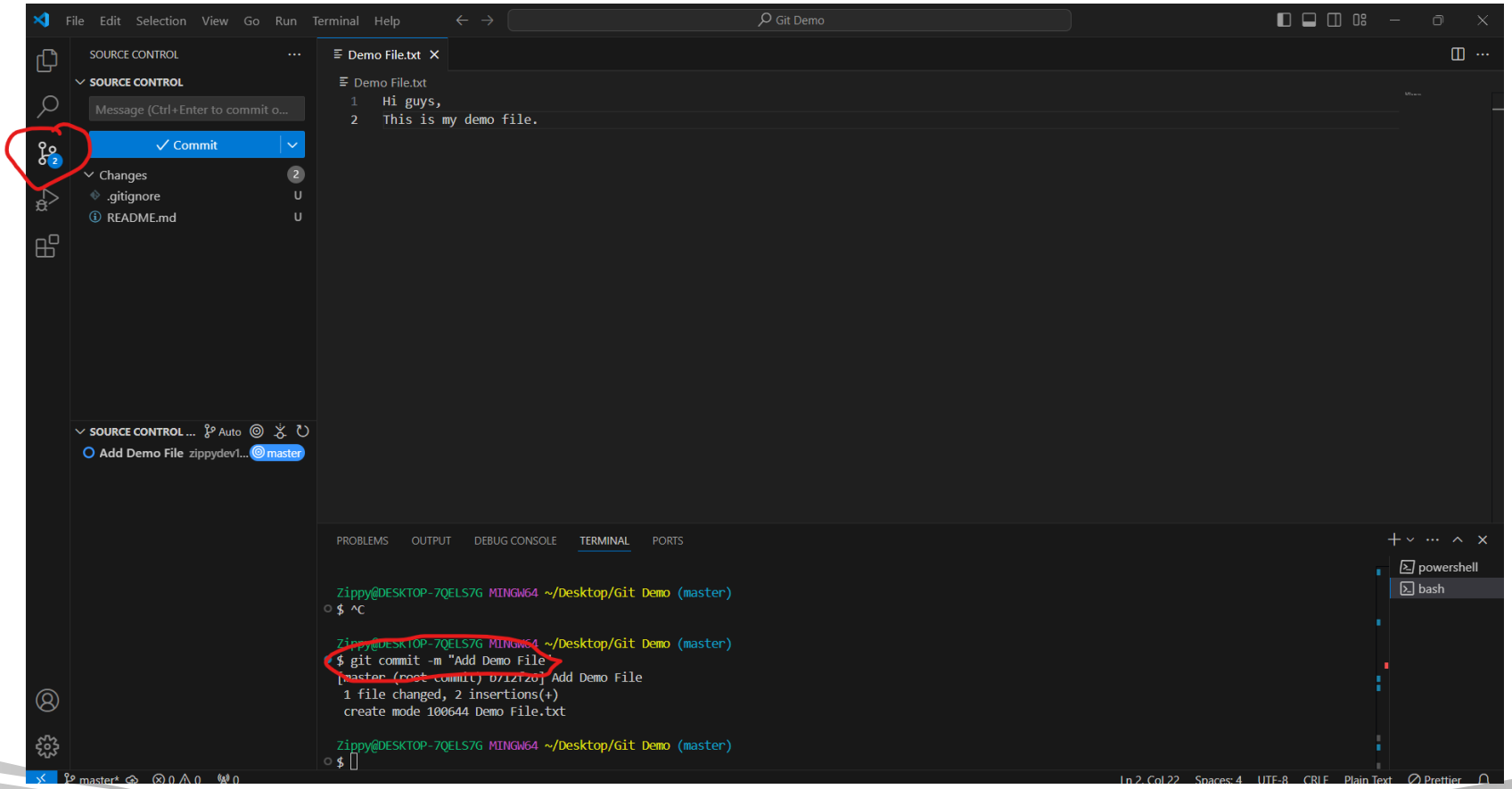
Tracking a File in Git: A Step-by-Step -Committing File

© Zippy Nadel 052-7102588

דוגמה:

הוספת הקובץ לשלב ה- Committed באמצעות הפקודה git Commit

ניתן לראות בטאב של ה- Source Control שהקובץ לא נמצא ב- Changes (למרות שכמובן הוא קיים במאגר) וזאת מאחר והקובץ נשמר בהסטוריית גרסאות של גיט וכרגע הוא לא מזוהה ע"י גיט כקובץ שהשתנה ואינו שמור.



Committed File- Example

© Zippy Nadel 052-7102588

מה קורה אם נשנה את הקובץ?

אם נפתח את Demo File ונערוך אותו, גיט יזהה את השינוי ויעביר אותו למצב Modified

ניתן לראות זאת בעזרת git status שם גיט יראה שהקובץ השתנה, אבל הוא עדיין לא שמור בגרסה מעודכנת. וכן ב- Source Control עם הסימון M (Modified) לידו.

כדי לשמור את הגרסה החדשה, יש לבצע שוב את התהליך –

יש להשתמש ב- git add כדי להעביר את הקובץ ל- Staging Area ואז יש לבצע commit נוסף כדי לשמור את השינוי.

Tracking a File in Git: A Step-by-Step -Modified File

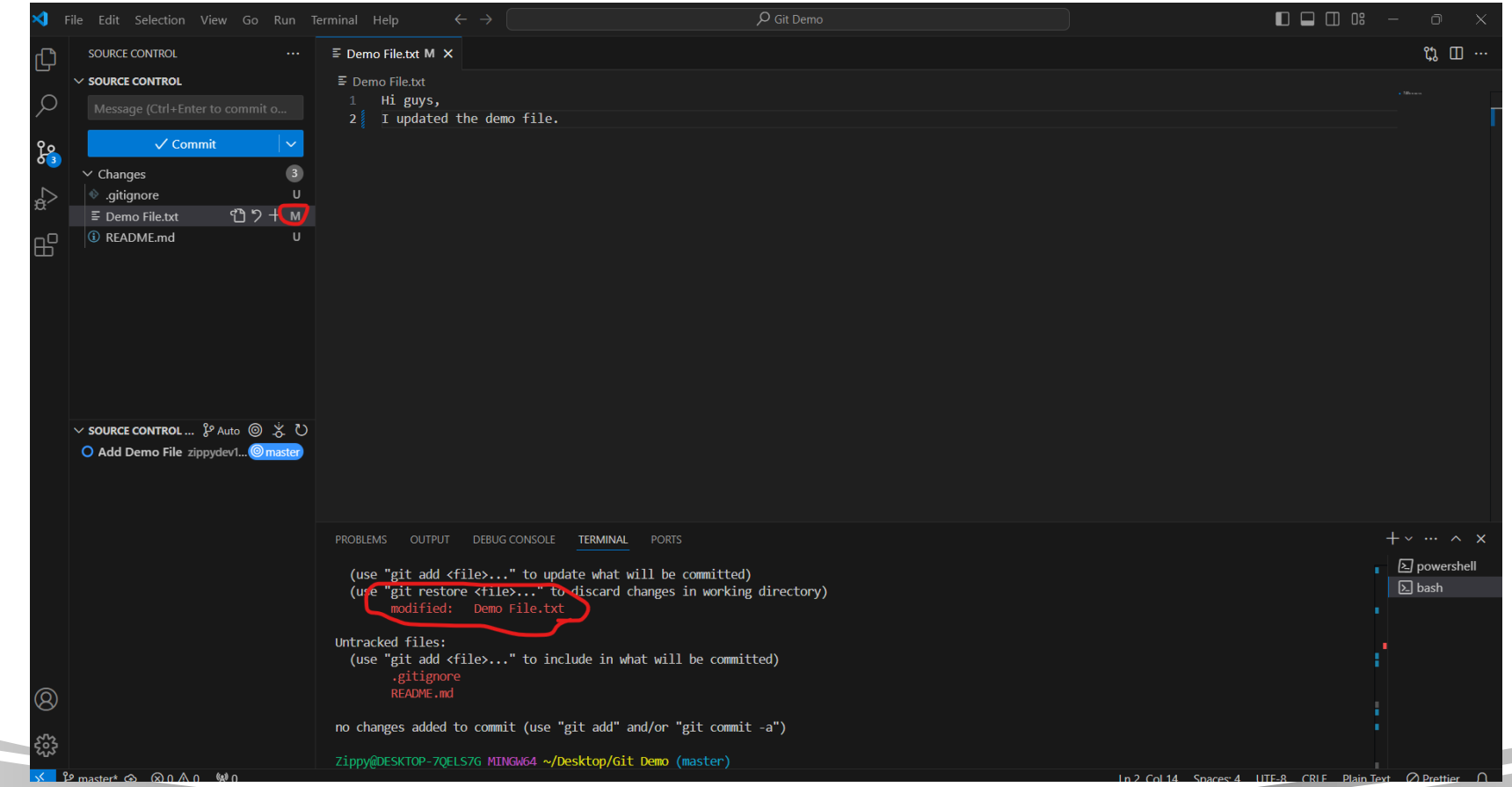
© Zippy Nadel 052-7102588

דוגמה:

קובץ שכבר נמצא בהיסטורית גרסאות של גיט- והשתנה.

ניתן לראות בטאב של ה- Source Control שהקובץ מסומן ב- M

וכן git status מציג אותו כ- modified



Modified File- Example

© Zippy Nadel 052-7102588

סיכום המסלול בקצרה:

1. **Untracked** -נוצר קובץ חדש וגיט לא עוקב אחריו.
2. **Staged** -שימוש ב- **git add** על מנת שגיט יוכל לעקוב אחר הקובץ.
3. **Committed** - שימוש ב- **git commit** על מנת לשמור את הקובץ בהיסטוריה של הפרויקט.

כל אחד מהשלבים הללו הוא חלק חשוב במעקב וניהול גרסאות, ומספק לגיט מידע מדויק על מצב הקבצים לאורך כל הדרך.

בכל שלב ניתן לעקוב אחר מצב הפרויקט באמצעות הפקודה **git status**

Tracking a File in Git: A Step-by-Step - Summery

© Zippy Nadel 052-7102588

Git Changes

הפקודה git status יכולה להציג רשימה של Git Changes- קבצים שעברו שינוי כלשהו מאז השמירה האחרונה במאגר.

ברשימה זו גיט מזהה ומסווג את הקבצים לפי מצבם הנוכחי, כדי לעזור להבין בדיוק מה השתנה בפרויקט.

(קבצים שהשתנו מוצגים באדום וקבצים שנוספו ל- Stage מוצגים בירוק).

כמו כן ניתן לראות את השינויים ב- Source Control.

קטגוריות השינויים בגיט:

* Untracked files- קובץ חדש שנוסף.
* Modified files- קובץ ששונה.
* Deleted files- קובץ שנמחק.

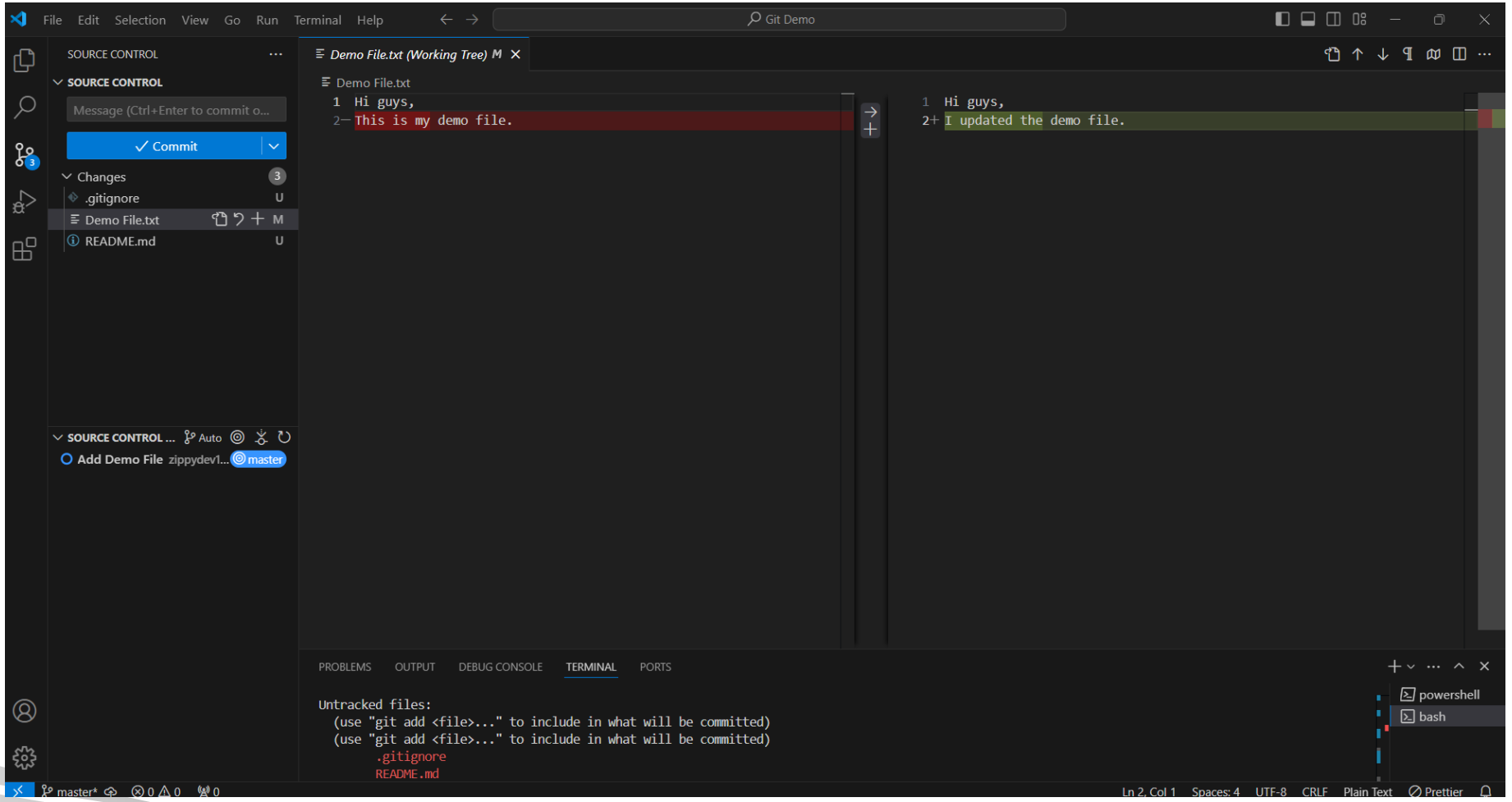
כל שינוי בקובץ שרוצים לשמור בהיסטורית הגרסאות- יש לעבור את התהליך של git add & git commit.

Git Changes

© Zippy Nadel 052-7102588

VS Code (בדומה לעורכי קוד וכלים נוספים) מספק דרך לעקוב אחר מהות השינויים שנעשו בקבצים.

כאשר נכנסים לקובץ שהשתנה דרך ה- Source Control ניתן לראות באדום את הגרסה הישנה (לפני השינוי) ובירוק את הגרסה החדשה (אחרי השינוי שנעשה).



Git Changes -Example

© Zippy Nadel 052-7102588

תרגיל 3

פקודות git בסיסיות- ניהול שינויים לוקליים:

1. הוסיפי קובץ חדש בשם File Demo למאגר המקומי הקיים.
2. שמרי את הקובץ בהסטוריית הגרסאות.
3. ערכי שינויים בקובץ ושמרי את הקובץ המעודכן בהסטוריית הגרסאות.
4. צרי מאגר חדש (לוקלי ומרוחק) עבור פרויקט FE. פתחי פרויקט React חדש.
(הפרויקט יקרא בשם your-app-name-frontend) ושמרי אותו בהסטורית גרסאות. (ל- Commit הראשון יש לקרוא Initial Commit).
5. צרי מאגר חדש (לוקלי ומרוחק) עבור פרויקט BE. פתחי פרויקט NET WEB API. חדש
(הפרויקט יקרא בשם your-app-name-backend) ושמרי אותו בהסטורית גרסאות. (ל- Commit הראשון יש לקרוא Initial Commit) שימי לב להוסיף קובץ gitignore וקובץ readme(לא נוצר אוטומטי).

בשלב זה- אין צורך לערוך שינויים נוספים בפרויקטים הראשוניים שנוצרו ואין צורך להעלות את השינויים למאגר המרוחק

Basic Git Commands-Managing Local Changes-Exercise

© Zippy Nadel 052-7102588

Git Push- שליחת שינויים למאגר המרוחק

ברגע שנוצר commit של השינויים במאגר המקומי, יש לשלוח את השינויים למאגר המרוחק כדי שגם אחרים יוכלו לראותם ולעבוד איתם. לשם כך יש להשתמש בפקודת push:

git push origin <Your Branch Name>

מה קורה בפועל?

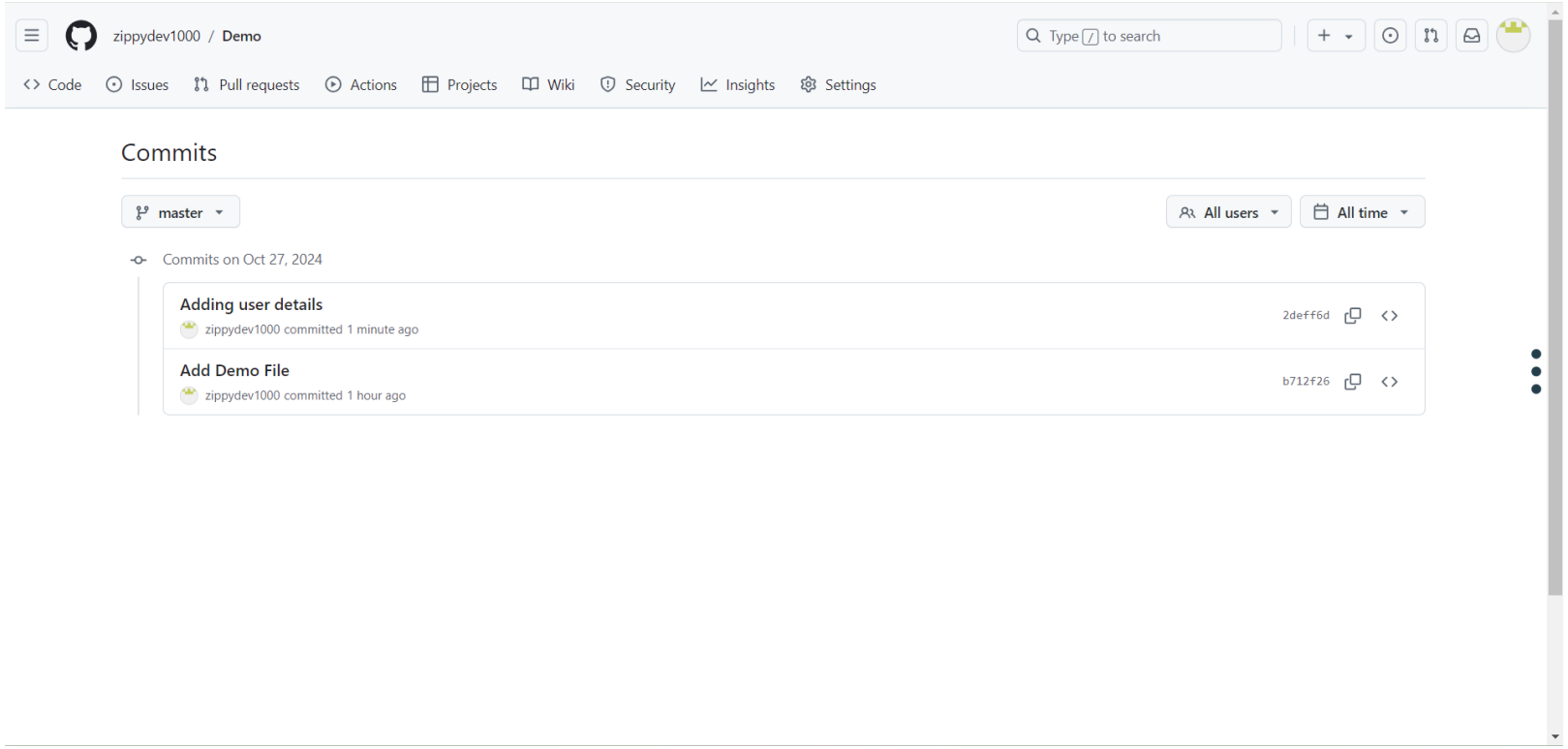
שליחת השינויים – כל השינויים שנשמרו ב- commits נשלחים לענף (branch) המתאים במאגר המרוחק. השינויים זמינים כעת למשתמשים אחרים שיש להם גישה למאגר המרוחק, והם יכולים לראות את העדכונים שבוצעו בקבצים.

הערה: כל נושא ה- Branches יורחב בהמשך, כרגע העבודה מתבצעת רק על ה- Branch הדיפולטי שנקרא master/main

Syncing with Remote Repository - Push

© Zippy Nadel 052-7102588

כאשר עושים Push ניתן לראות בגיטהאב את ה- Commit שנוסף ל- Branch.



Git Push -Example